Washington State University Vancouver

Systems Programming - CS 360

Assignment 5

Instructor:
Ben McCamish

Overall Assignment - 100 points

Write a program (in C) called `assignment5.c` targeted at the Linux platform. This program must fork 5 child processes, each representing one philosopher. Using shared memory and/or semaphores, have each philosopher repeat a cycle of eating and thinking until all philosophers have eaten for at least eat_time seconds (at which point a philosopher will "leave the table"... terminate). Use the IPC system calls described in the lecture slides (System V Semaphores).

There are 5 chopsticks at the table (one between each pair of adjacent philosophers). In order to eat, a philosopher must acquire both the chopsticks adjacent to them. If they cannot, they are made to wait until they can. The wait time does not count as thinking time. Once they have a pair of chopsticks, they eat for the expected period of time and then replace (release) the chopsticks onto the table and begin their period of thinking. Each philosopher repeats this process until they have eaten for a total of eat_time seconds or more.

There is a routine named "randomGaussian" available to you in the handout that you will use to simulate the random amounts of time that a philosopher eats and thinks. The arguments to this function are provided through command-line arguments (see below). Obviously, if randomGaussian returns a negative time value, treat it as zero. Simulate the eating or thinking time by calling sleep() for the requisite number of seconds. You should embed `randomGaussian()` into your code. This means, that all I should need to compile is the `assignment5.c` program and run the executable produced. No additional files should be submitted or required. Example compilation statement:

```
gcc -o assignment5 assignment5.c -lm
```

You can either represent the chopsticks in a shared IPC memory segment and use a semaphore to "authorize" mutually exclusive access to the shared memory segment, or you can define an array of IPC semaphores, each representing one chopstick.

You can execute a implementation of this program in the handout (use -d on the command line to turn on more output). You do not need to match its output exactly, but print enough information about what the philosophers are doing to demonstrate your implementation is functioning correctly. Note, that does not prevent you from installing as much debugging output as you think you need. If your output is too cluttered or does not contain enough information to see what is going on, then you will not receive credit for the assignment.

Program Interface (Required)

```
./assignment5 <eat_time> <rand_mean> <rand_stddev> <philo_count>
```

Your program will receive 4 command-line arguments, each guaranteed to be an integer. The first argument is the target eat time of each philosopher mentioned earlier. The second and third arguments are the arguments you must pass to the provided randomGaussian function. The last argument is the number of philosophers to simulate. You may ignore this if you do not wish to implement the extra credit feature (see below).

Your program must only print to stdout in three scenarios: when a philosopher is thinking, eating, and leaving the table. The eating and thinking print statements must precede their corresponding sleep() calls. The print statements must be formatted like so:

```
// thinking
printf("philosopher %d is thinking for %d seconds\n", id, think_time);
...
// eating
printf("philosopher %d is eating for %d seconds\n", id, eat_time);
...
// leaving
printf("philosopher %d ate for %d seconds and is leaving the table\n", id, total_eat_time);
```

The _time variables are assumed to be the result of calling randomGaussian, and should be passed to a corresponding sleep call. The id variable is the unique identifier of each philosopher and must be within the range [0, philo_count - 1].

Specifications and Restrictions

- (80 points) Program must work on the lab machines, including the specifications above.
- (20 points) Must be robust, including error catching. You must catch errors and print out to stderr an appropriate error message containing the `errno` and the message produced by that error. This means you will need to use `errno.h` and `string.h`, libraries at least.
- (Note) If you make use of shared memory, use the `shmget()` and `shmat()` system calls. `shmctl()` can be used to remove the shared memory segment you create.
- (Note) To create and manipulate semaphores, use `semget()` and `semop()`. `semctl()` is used to remove a semaphore set.
- (Note) The Autolab tests will try their best to thoroughly test your program. However, it's not uncommon for it to give false passes in some categories like deadlocking, eating order, and others. As a precaution, make sure to test your program thoroughly before submitting.
- (Required) Clean up the IPC artifacts you create so that you don't leave shared memory and semaphore IDs cluttering up the lab systems.
- (Extra credit) Allow your program to work with an arbitrary number of philosophers. The philosopher count is passed as the fourth command-line argument.

What to turn in (on Autolab):

- assignment5.c (no header files)